Word Frequency Analyzer

Design Report & Technical Documentation

Modugu Akhilesh Kumar | Roll No : 240656

1. Overview

The Word Frequency Analyzer is a command-line C++ program that reads large text files in fixed-size buffered chunks, builds per-version word-frequency maps, and answers three types of queries over them.

The program is driven entirely by command-line arguments. A user specifies one or two input files, assigns each a *version name*, provides a buffer size (256–1024 KB), and selects a query. File I/O is encapsulated in a custom `FileReader` class, keeping peak memory usage bounded regardless of file size.

Tokenisation is performed per-chunk. A `leftover` string carries any partial word at a chunk boundary into the next iteration so no token is lost. All tokens are lowercased and counted in an `unordered_map`. The three concrete query types all derive from an abstract base class `Query` and are dispatched polymorphically through a `Query*` pointer in `main()`.

2. Data Flow

Stage	Component	Output
Argument parsing	`main()`	filePath, versionName, queryType, ...
File ingestion	`FileReader`	Raw binary chunks (`string`)
Normalisation	`buildVersion()`	Lowercased chunk + leftover merge
Tokenisation	`tokenizeAndCount()`	`unordered_map<string,int>`
Storage	`versions` (global)	`unordered_map<string, unordered_map<string,int>>`
Query dispatch	`Query*` (polymorphic)	Printed result to stdout

Table 1: End-to-end data flow

3. Template Function — `printContainer<T>`

`printContainer` is a generic utility function that iterates over any container whose elements expose `.first` and `.second` members and prints each pair to stdout.

```
template<typename T>
```

```
void printContainer(const T& container) {
    for (const auto& p : container)
        cout << p.first << " " << p.second << endl;
}
```

The template parameter **T** can be any iterable whose value type has `first` and `second` fields — for example a `vector<pair<string,int>>` or an `unordered_map<string,int>`. This makes it reusable for printing frequency maps or top-K result vectors without writing separate print loops for each use case.

4. Class Descriptions

4.1 FileReader

Responsibility: Opens a binary file and delivers its contents as a sequence of fixed-size string chunks via `readChunk()`.

- `vector<char> buffer` — allocated once at construction via `resize()` and reused on every `readChunk()` call, avoiding repeated heap allocation per chunk.
- `readChunk(string& output)` — fills `output` with up to `bufferSize` bytes using `assign()`; returns `false` at EOF.
- **Constructor** — throws `invalid_argument` if `bufferKB` is outside [256, 1024] and `runtime_error` if the file cannot be opened.
- **Destructor** — explicitly closes the file handle if still open.

4.2 Query (Abstract Base Class)

Responsibility: Declares the pure-virtual method `execute()` that all concrete query types must implement. The abstract base allows `main()` to dispatch any query through a single `Query*` pointer without knowing the concrete type at compile time, following the Open/Closed Principle.

4.3 WordQuery : public Query

Responsibility: Looks up the count of a single word in a named version.

- Checks that the version exists in the global `versions` map.
- Looks up the word in the version's frequency map.
- Prints the version name and count, or `Word not found` if absent.

4.4 TopKQuery : public Query

Responsibility: Returns the K most frequent words in a named version, sorted in descending order of frequency.

- Copies the frequency map into a `vector<pair<string,int>>` for sorting.
- Sorts the full vector in descending order of count using `std::sort` with a lambda comparator — $O(N \log N)$ where N is the vocabulary size.

- Prints the first min(k, *vocabsize*) entries.

4.5 DiffQuery : public Query

Responsibility: Computes the signed difference in a word's count between two named versions.

- Validates that both version names exist in the global map.
- Reads each version's count for the word; defaults to 0 if the word is absent.
- Prints `c2 - c1`: positive means the word grew, negative means it shrank.

5. Key Free Functions

5.1 tokenizeAndCount()

Iterates over every character in a chunk in $O(C)$. Alphanumeric characters accumulate into a local `token` string. Any non-alphanumeric character flushes the current token into the frequency map and resets the accumulator. A trailing token after the loop ends is also flushed.

5.2 buildVersion() — Two Overloads

The program provides two overloads:

```
// Core overload
void buildVersion(const string& filePath,
                  const string& versionName,
                  int bufferSize);

// Verbose overload -- delegates to core, then prints a
   confirmation
void buildVersion(const string& filePath,
                  const string& versionName,
                  int bufferSize, bool verbose);
```

The **core overload** handles all I/O and tokenisation: it reads chunks, lowercases each one, prepends the previous `leftover`, strips any trailing partial token back into `leftover`, tokenises the clean chunk, and finally `moves` the completed map into the global `versions` store.

The **verbose overload** delegates entirely to the core and, if the `verbose` flag is `true`, prints a confirmation message. This keeps the core overload free of any conditional branches.

6. Global State

A single global map stores all loaded versions:

```
unordered_map<string, unordered_map<string,int>> versions;
```

The outer map keys are version names; the inner maps hold word-to-count frequencies. All query classes access this map directly. Every version loaded during a run remains in memory for the lifetime of the process.

7. Assumptions & Observations

7.1 Assumptions

- Files are plain text in ASCII or UTF-8; only ASCII alphanumeric characters form tokens.
- Buffer size is always supplied on the command line and is within [256, 1024] KB.
- For `word`/`top` queries, exactly one `-file`/`-version` pair is given; for `diff`, exactly two pairs are given.
- The query word is lowercased by `main()` before being passed to any query object.
- All versions fit in RAM simultaneously; no eviction mechanism exists.

7.2 Observations

Area	Observation
`printContainer<T>`	Generic and reusable for any pair-iterable container. Currently not called inside any query class — it can replace the print loops in `TopKQuery` or `WordQuery` directly.
`buildVersion` overloads	Clean use of function overloading: core logic stays free of conditional branches; the verbose flag is handled entirely in the second overload.
`TopKQuery` sort	Sorts the entire vocabulary ($O(N \log N)$) on every query call. `std::nth_element` would reduce this to $O(N) + O(k \log k)$ for large vocabularies.
Global `versions` map	All versions remain in RAM for the process lifetime. Freeing a version after its query completes would reduce peak memory.
Error output	Some error messages are sent to `cout`. Routing all errors to `cerr` would cleanly separate output and error streams.
Build timing	The core overload does not measure or report build time. Adding `chrono` timing would help compare the effect of different buffer sizes.

Table 2: Observations and suggested improvements